



UNIVERSIDAD DE LOS LAGOS

INFORME BREVE — LABORATORIO N°1

FALACIAS DE LOS SISTEMAS DISTRIBUIDOS

DEPARTAMENTO DE CIENCIAS DE LA INGENIERÍA
INGENIERÍA CIVIL EN INFORMÁTICA
SISTEMAS DISTRIBUIDOS
UNIVERSIDAD DE LOS LAGOS, CHILE

Álvaro Andler
Elías Soto

2 de septiembre de 2026



5 **UNIVERSIDAD ACREDITADA**
Septiembre de 2021 - Septiembre de 2026
Gestión Institucional - Docencia de Pregrado
Investigación - Vinculación con el Medio
AVANZADA

www.ulagos.cl

1. Entorno y método

El laboratorio se realizó en **Docker Desktop en Windows**, utilizando dos contenedores conectados a la red `sd_net`: un cliente y un servidor TCP en el puerto 5000. Primero se obtuvo una línea base mediante `ping` e `iperf3`. Luego se ejecutaron 100 llamadas remotas secuenciales con `cliente.py`. Sobre la interfaz `eth0` del cliente se inyectaron retardos de 50, 200 y 500 ms con `tc netem`, repitiendo las 100 llamadas en cada condición. Finalmente, se evaluó pérdida de paquetes de 1 %, 5 %, 20 % y 100 % y no fue necesario utilizar el Plan B.

2. Resultados

La línea base entregó un RTT promedio de **0,068 ms** y un throughput de **22,2 Gbit/s**. Los resultados de latencia se resumen en la Tabla 1.

Tabla 1: Resultados con latencia inyectada para 100 llamadas.

Latencia (ms)	Total (s)	Promedio (ms)	Máximo (ms)
0	0,011	0,1	0,3
50	5,034	50,3	50,7
200	20,035	200,2	200,6
500	50,035	500,3	500,6

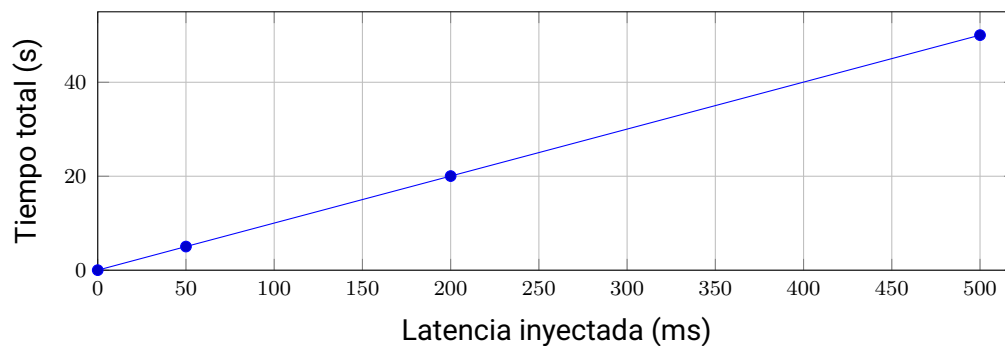


Figura 1: Tiempo total de 100 llamadas según la latencia inyectada.

3. Observado vs. esperado

Al aumentar la latencia, el tiempo total también aumentó de forma proporcional. Con 50 ms se obtuvieron 5,034 s y con 500 ms, 50,035 s, valores cercanos a lo esperado para 100 llamadas secuenciales.

En las pruebas con pérdida de paquetes el efecto fue distinto: el cliente continuó funcionando, pero el rendimiento disminuyó considerablemente a medida que aumentaba la pérdida.

Tabla 2: Efecto de la pérdida de paquetes.

Pérdida	Throughput	Total	Observación
1 %	7,13 Gbit/s	0,216 s	La comunicación continuó con una disminución del rendimiento.
5 %	223 Mbit/s	1,468 s	El throughput disminuyó considerablemente y aumentó el tiempo total.
20 %	604 Kbit/s	5,016 s	La comunicación se volvió muy lenta debido a la alta pérdida.
100 %	–	–	El cliente no pudo establecer la comunicación y mostró Errno 113: No route to host.

Los resultados coinciden con lo esperado: una mayor latencia aumenta el tiempo de respuesta y una mayor pérdida reduce el rendimiento. La principal diferencia ocurrió con 100 % de pérdida, donde el cliente terminó con `No route to host` en vez de quedar esperando indefinidamente.

4. La falacia que asumimos

La principal falacia observada fue “**la red es confiable**”, una de las falacias vista en clases.

En `cliente.py`, el cliente espera la respuesta mediante `f.readline()` y, por defecto, no tiene un tiempo máximo de espera definido. Esto supone que el servidor siempre responderá, situación que deja de cumplirse cuando ocurre una falla de red.

Una forma de corregirlo es utilizar un **timeout**. Por ejemplo, con `TIMEOUT_S=3` el cliente puede detectar que no recibió respuesta y terminar de forma controlada, en lugar de esperar sin un límite definido.

5. Relación con la investigación

Esta falacia también puede afectar a la aplicación estudiada para la Evaluación 1. Si uno de sus componentes depende de otro a través de la red, una falla o una respuesta demasiado lenta puede afectar el funcionamiento del sistema completo.

Por esto, la aplicación debería considerar situaciones donde un servicio no responda, utilizando mecanismos como tiempos de espera, manejo de errores y reintentos cuando corresponda.